



CI/CD

Introduction to CI/CD



Outline

1

Jenkins Advance

1

Jenkins Advance



Jenkins

Jenkins bukan hanya sekadar menjalankan job build/test. Dengan fitur lanjutan, Jenkins bisa menangani **pipeline kompleks**, **deployment multi-environment**, dan **automasi infrastruktur** dengan **kontrol yang sangat tinggi**.



Pipeline as Code

Pipeline as Code memungkinkan kamu menulis alur CI/CD dalam file teks (**Jenkinsfile**) yang disimpan bersama kode sumber atau di dalam konfigurasi project

Pipeline as Code

Gaya	Declarative	Scripted
Sintaks	Tersruktur, deklaratif	Imperatif, bebas seperti Groovy
Mudah dipahami	✅ Ya	❌ Cenderung kompleks
Kontrol logika	Terbatas	Sangat fleksibel
Direkomendasikan	✅ Oleh Jenkins	⚠️ Hanya untuk kasus kompleks

Declarative

```
pipeline {
  agent any
  environment {
    ENV = 'staging'
  }
  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/example/repo.git'
      }
    }
    stage('Build') {
      steps {
        sh './gradlew build'
      }
    }
    stage('Test') {
      steps {
        sh './gradlew test'
      }
    }
  }
}
```

Kelebihan

- Lebih aman (validasi struktur)
- Lebih mudah dibaca pemula
- Integrasi mudah dengan plugin Jenkins modern

Scripted

```
node {  
  stage('Checkout') {  
    git 'https://github.com/example/repo.git'  
  }  
  
  stage('Build') {  
    sh './gradlew build'  
  }  
  
  stage('Test') {  
    if (env.BRANCH_NAME == 'main') {  
      sh './gradlew test'  
    } else {  
      echo 'Skip test on non-main branch'  
    }  
  }  
}
```

Kelebihan

- Kontrol penuh terhadap alur, logika, kondisi
- Cocok untuk pipeline yang sangat kompleks
- Bisa digunakan untuk looping, dynamic stages, dsb

Modularisasi & Reusability dengan Shared Libraries

Apa tujuan dari Shared Lib

Tujuan Shared Library

Ketika banyak proyek Jenkins memiliki logika pipeline yang mirip, misalnya:

- Build Java app
- Linting
- Docker build & push
- Deploy ke Kubernetes

akan sangat **tidak efisien** jika semua Jenkinsfile menyalin kode yang sama berulang-ulang.

Solusinya: **Shared Library**

- Centralisasi semua fungsi umum ke dalam satu tempat
- Digunakan kembali lintas proyek

Jenkins Parallel build & Matrix Build

Apa itu Paralel Stage

Parallel stage adalah fitur di Jenkins Pipeline yang memungkinkan Anda menjalankan beberapa proses secara **bersamaan (paralel)** daripada berurutan.

Tujuan:

- Mengurangi total waktu build
- Menjalankan berbagai jenis test (unit, integration, e2e) secara bersamaan
- Melakukan linting dan security scan secara paralel

Contoh Parallel Stage



```
stage('Test') {  
  parallel {  
    unitTests: {  
      sh './gradlew test'  
    }  
    integrationTests: {  
      sh './gradlew integrationTest'  
    }  
    linting: {  
      sh 'npm run lint'  
    }  
  }  
}
```

Apa itu Matrix Build

Matrix Build adalah strategi di mana Jenkins menjalankan build **dengan kombinasi variabel** tertentu, misalnya:

- OS (Linux, Windows, Mac)
- Java version (8, 11, 17)
- Arsitektur CPU (x86, arm64)
- Environment (staging, prod)

Tujuan:

- Menguji kompatibilitas aplikasi pada berbagai konfigurasi
- Mendeteksi bug environment-specific lebih awal
- Automasi pengujian kompleks tanpa menulis pipeline berulang

Contoh Matrix Build

Jenkins akan menjalankan total **4 kombinasi:**

- JDK 8 on Linux
- JDK 8 on Windows
- JDK 11 on Linux
- JDK 11 on Windows

```
pipeline {
  agent none
  matrix {
    axes {
      axis {
        name: 'JDK'
        values: '8', '11'
      }
      axis {
        name: 'OS'
        values: 'linux', 'windows'
      }
    }
  }
  agent any
  stages {
    stage('Build') {
      steps {
        echo "Building on JDK ${JDK}, OS ${OS}"
        // Implement build logic here
      }
    }
    stage('Test') {
      steps {
        echo "Running tests on JDK ${JDK}, OS ${OS}"
      }
    }
  }
}
```

Komparasi



Aspek	Matrix Build	Parallel Stage
Konfigurasi kombinasi otomatis		
Harus ditentukan manual		
Cocok untuk kombinasi variabel		
Cocok untuk proses berbeda		
Kompleksitas	Lebih tinggi	Lebih sederhana

Kapan menggunakan antara keduanya

Gunakan **Parallel** jika:

- Test-nya jenisnya beda (unit, integration, e2e)
- Proses tidak terkait environment

Gunakan **Matrix** jika:

- Anda ingin mengecek *combinatorial compatibility* (misalnya kombinasi OS + JDK)
- Cocok untuk proyek lintas platform atau arsitektur

Tips:

- Pastikan worker Jenkins (agent/slave) **mendukung environment** yang ingin diuji
- Hati-hati terhadap limitasi resource saat menggunakan terlalu banyak parallel/matrix

Jenkins Credential dan Secret Manager

Apa itu Jenkins Credential dan Secret Manager



Jenkins membutuhkan informasi sensitif seperti password, API token, SSH key, dan lain-lain untuk berinteraksi dengan sistem lain (GitHub, Docker Registry, Cloud Provider, dsb).

Untuk alasan keamanan dan modularitas, Jenkins menyediakan Credential Management untuk menyimpan dan mengelola data sensitif ini secara aman.

Tipe Credential Jenkins



Tipe Credential

Kegunaan

Username with Password

Autentikasi ke layanan (e.g. Git, Docker Hub)

Secret Text

Token API atau access key

SSH Username with Private Key

Akses ke server melalui SSH

Certificate

Sertifikat digital untuk TLS

Secret File

Upload file rahasia seperti `.p12` atau `service-account.json`

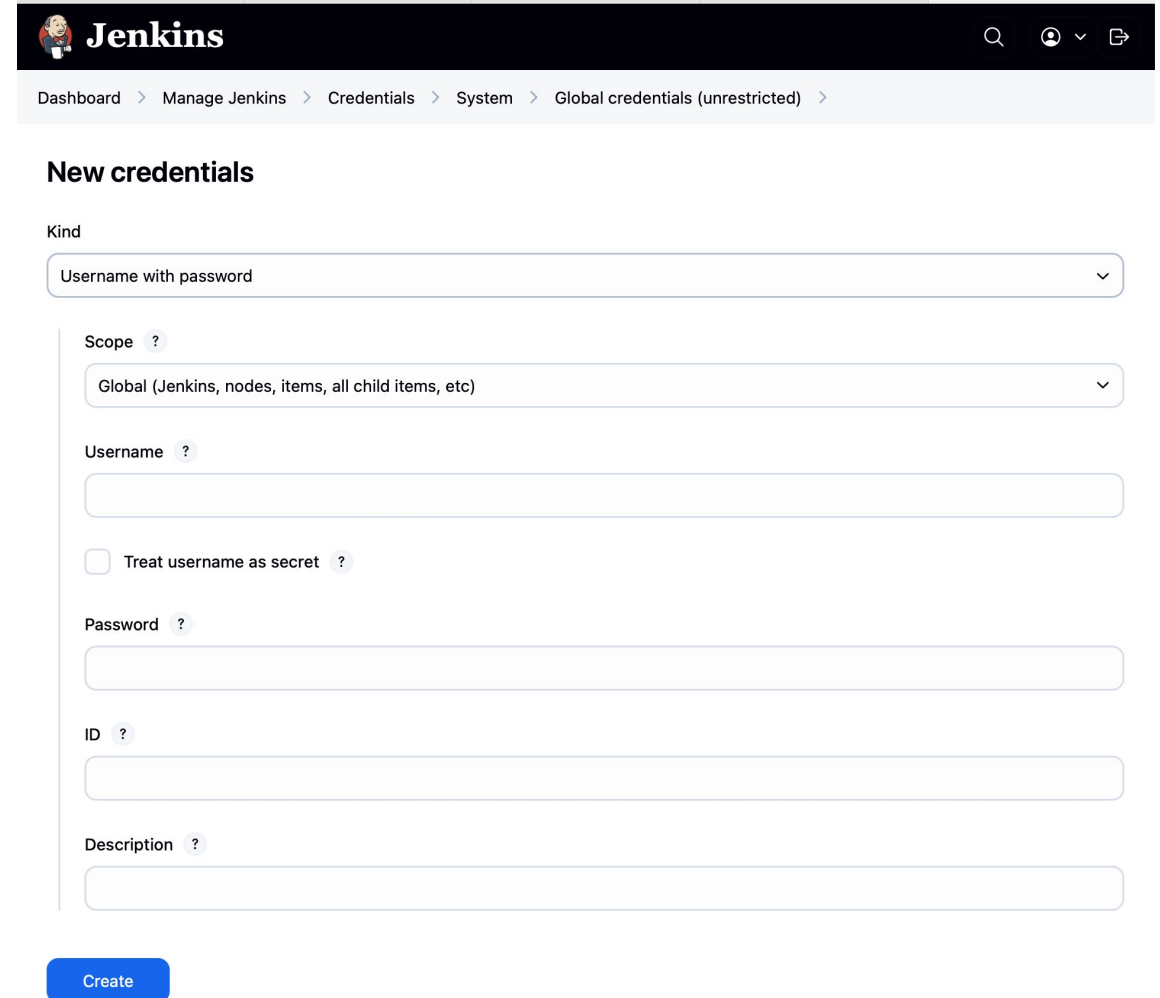
Menambah Credential Baru

Langkah-langkah:

1. Buka [Jenkins](#) → [Manage Jenkins](#) → [Credentials](#)
2. Pilih **(global)** atau scope tertentu (folder/job)
3. Klik [Add Credentials](#)
4. Pilih jenis credential → Masukkan data → Simpan

Best Practice:

- ✓ Gunakan scope sekecil mungkin (folder/job)
- ✓ Gunakan deskripsi yang jelas



The screenshot shows the Jenkins web interface for creating a new credential. The breadcrumb trail is: Dashboard > Manage Jenkins > Credentials > System > Global credentials (unrestricted). The form is titled 'New credentials' and includes the following fields:

- Kind:** A dropdown menu with 'Username with password' selected.
- Scope:** A dropdown menu with 'Global (Jenkins, nodes, items, all child items, etc)' selected.
- Username:** A text input field.
- Treat username as secret:** An unchecked checkbox.
- Password:** A text input field.
- ID:** A text input field.
- Description:** A text input field.

A blue 'Create' button is located at the bottom of the form.

Credential Scope

Scope	Penjelasan
Global	Tersedia untuk seluruh job
System	Hanya untuk internal plugin Jenkins
Folder / Domain	Hanya tersedia untuk job dalam folder/domain tertentu

```
pipeline {  
  agent any  
  environment {  
    GITHUB_TOKEN = credentials('github-token-id')  
  }  
  stages {  
    stage('Clone Repo') {  
      steps {  
        sh 'git clone https://$GITHUB_TOKEN@github.com/myorg/myrepo.git'  
      }  
    }  
  }  
}
```


Tips Keamanan

- ❌ Jangan hardcode secret dalam `Jenkinsfile`
- ✓ Gunakan `withCredentials{}` block
- 🔒 Gunakan **Vault Plugin** atau integrasi dengan **HashiCorp Vault** untuk skenario skala besar
- 🔄 Putar credential secara berkala
- 📜 Audit penggunaan credential (melalui log atau plugin audit)

Jenkins Trigger

Apa itu Jenkins Trigger ?

Jenkins memiliki berbagai mekanisme pemicu (trigger) untuk mengeksekusi job secara otomatis berdasarkan suatu kondisi. Trigger lanjutan membuat pipeline lebih responsive, event-driven, dan efisien, khususnya pada sistem berskala besar dan tim kolaboratif.

Apa itu Jenkins Trigger ?

Jenis Trigger	Penjelasan
SCM Polling	Mengecek perubahan berkala dari repository
Webhook	Build otomatis ketika ada push/pull request
Parameterized Trigger	Build berdasarkan parameter tertentu
Upstream/Downstream Trigger	Build job lain saat job selesai
Cron Timer	Penjadwalan mirip cron
Manual Approval	Menunggu approval dari user
External Trigger	Trigger dari API, CLI, plugin
GitHub/GitLab Integration	Khusus integrasi dengan platform git
Remote Trigger	Build dari sistem eksternal menggunakan token URL

SCM Polling

Jenkins melakukan pengecekan berkala ke Git repository.
Jika mendeteksi perubahan SHA terakhir, maka pipeline dijalankan.

```
pipeline {
  triggers {
    pollSCM('H/5 * * * *') // Cek setiap 5 menit
  }
  stages {
    stage('Build') {
      steps {
        echo 'SCM Polling triggered build'
      }
    }
  }
}
```

SCM Polling

Kelebihan:

- Mudah dikonfigurasi langsung dari Jenkins
- Tidak butuh konfigurasi di repo Git

Kekurangan:

- Boros resource (terutama pada skala besar)
- Tidak real-time (delay tergantung interval polling)
- Tidak efisien untuk banyak repository

Webhook

Cara Kerja:

- Repository Git mengirim notifikasi (event) ke Jenkins saat ada perubahan
- Real-time trigger, tidak menunggu jadwal

Cara Konfigurasi:

1. Di Jenkins, **aktifkan GitHub/GitLab webhook trigger**
2. Catat endpoint webhook Jenkins:

Kelebihan:

- Real-time responsif
- Ringan dan efisien
- Skalabel untuk banyak repo/proyek

Kekurangan:

- Butuh konfigurasi di sisi Git (repo)
- Jenkins harus bisa diakses publik atau gunakan tunnel (e.g., cloudflared/ngrok)
- Rentan terhadap perubahan IP jika tanpa DNS tetap